NATIONAL UNIVERSITY

NATIONAL UNIVERSITY
COLLEGE OF COMPUTING AND
INFORMATION TECHNOLOGIES

Laboratory Activity 4
Authentication and Authorization

Name	Marzan, Joshua M.
Section	INF 233
Date	19 October 2026

Objectives

- Explain the difference between authentication and authorization.
- Implement user registration using password hashing with Bcrypt.
- Implement user login using JSON Web Tokens (JWT).
- Protect RESTful API endpoints using token-based authentication.
- Implement Role-Based Access Control (RBAC) for Bulldogs Exchange users.
- Apply permissions based on user roles.

Materials Needed:

- Visual Studio Code
- Node.js (LTS Version)
- MongoDB Community Server or MongoDB Atlas
- Postman/Insomnia REST

Prerequisite(s):

- Completed Week 4 Laboratory Activity – RESTful API Design Standards
- Working Bulldogs Exchange backend
- Completed User Mongoose model
- Functional Product, Category, and Supplier APIs
- Basic understanding of HTTP requests and REST APIs

Submission Requirements:

- Filename: Surname_Activity4_CTADWEBL
- Save as PDF

Activity Instructions

Part 1: Running the Server
Open the Bulldogs Exchange backend project completed during Week 4. **Verify that (***Provide the Screenshot of your Backend Working***):** • The server starts successfully. • MongoDB connects successfully. • Existing RESTful APIs are functioning.

The Bulldogs Exchange backend (**marzan-server**) from Week 4 (Activity 3, RESTful API Design Standards) was started with `npm run dev`. The console confirms the MongoDB Atlas connection and that every Mongoose schema, including the updated userModel with its new `role` field, loaded without errors.
`> npm run dev`
`MongoDB connected`
`Models loaded: Product, Category, Supplier, User, Cart, Order, Review`
`Server running on http://localhost:5000`
• **The server starts successfully:** `GET http://localhost:5000/` returns `{ "success": true, "message": "BulldogEx Shop API is running", ... }`.
• **MongoDB connects successfully:** the console prints `MongoDB connected` (database `bulldogex-shop` on MongoDB Atlas).
• **Existing RESTful APIs are functioning:** `GET /api/v1/products` and `GET /api/v1/categories` from Activity 3 still return 200 OK with the pagination, filtering, sorting, and search features intact.
Screenshot below: VS Code with the terminal output of `npm run dev`.

Part 2: Preparing the User Model

In your User Model/Schema add the following fields: (*Provide Screenshot of your source code***):**

- **name**
- **email**
- **password**
- **role (**add a enum that only accepts “Customer” and “Admin”)

Decision on the "name" field: the User model already has **firstName** and **lastName** from Activity 2, and both are used throughout the project: `cart`, `order`, and `review` responses populate `"firstName lastName email"`, `seed.js` creates the sample accounts with them, the Activity 2/3 Postman collections send them, and the real accounts already saved in MongoDB Atlas (Juan, Maria, Jose, Ana, Carlo) already have these fields. Replacing them with a single `name` field would break all of that and the data already in Atlas, so **firstName + lastName together satisfy the "name" requirement** (a full name is `${firstName} ${lastName}`), and only the new field, `role`, was added.

- **email** and **password** already existed (Activity 2) and are unchanged.
- **role (new):** `String`, `enum: ["Customer", "Admin"]`, `default: "Customer"`, `required: true`. A client can never set this at sign-up; `signUp` always saves `role: "Customer"`, so the only way to create an Admin is by seeding one directly in the database (see `seed.js`), the same way a real

deployment creates its first Admin.

```
const userSchema = new mongoose.Schema({
  firstName: { type: String, required: [true, "First name is required"], trim: true },
  lastName: { type: String, required: [true, "Last name is required"], trim: true },
  email: { type: String, required: true, unique: true, lowercase: true, match: [...] },
  password: { type: String, required: [true, "Password is required"] },
  role: {
    type: String,
    enum: { values: ["Customer", "Admin"], message: "{VALUE} is not a valid role" },
    default: "Customer",
    required: true,
  },
}, { timestamps: true });
```

Existing accounts created before this activity did not have a `role` field. `seed.js` backfills them with `User.updateMany({ role: { $exists: false } }, { $set: { role: "Customer" } })` and also seeds one Admin account (`admin@bulldogex.nu.edu.ph`) for testing.

Screenshot below: components/models/userModel.js.

Part 3: Creating a User Registration and Login Controller

Do the following (*Provide Screenshot of your Source Code and Output, add more pages if necessary***):**

- Create a Registration function using bcrypt hashing algorithm inside your UserController
- Create a Login function using the jsonwebtoken library for session handling inside your UserController
- Import the Controller inside the UserRoutes
- Test the Registration and Login Feature using Postman/Insomnia REST

Authentication (this part) proves *who* is making the request; **authorization** (Parts 6–7) decides *what* that identity is allowed to do. The registration and login functions already existed from Activity 2/3 as `signUp` and `signIn` in `userController.js`; for this activity, `signUp` now always saves `role: "Customer"` and `signIn` embeds the user's `role` in the JWT, so the authorization middleware can check it without an extra database lookup.

Registration (bcrypt)

```
const signUp = async (req, res) => {
  const { firstName, lastName, email, password } = req.body;
  const existingUser = await User.findOne({ email });
  if (existingUser) return sendError(res, HttpStatus.CONFLICT, "Email already registered");

  const hashedPassword = await bcrypt.hash(password, SALT);
  const user = await User.create({ firstName, lastName, email, password: hashedPassword, role:
"Customer" });
  return sendSuccess(res, HttpStatus.CREATED, "User registered successfully.", { ...user, role:
user.role });
};
```

Login (jsonwebtoken)

```
const signIn = async (req, res) => {
  const { email, password } = req.body;
  const user = await User.findOne({ email });
  if (!user || !(await bcrypt.compare(password, user.password))) {
    return sendError(res, HttpStatus.UNAUTHORIZED, "Invalid email or password");
  }
  const token = jwt.sign({ id: user._id, email: user.email, role: user.role }, SECRET_KEY, {
expiresIn: "1d" });
  return sendSuccess(res, HttpStatus.OK, "Signed in successfully.", { token });
};
```

The controller was already imported in `userRoutes.js` since Activity 2 (`POST /api/v1/users` for registration, `POST /api/v1/sessions` for login). Both were tested in Postman (collection **Activity4-CTADWEBL**, folder 1): registering returns 201 with the password left out of the response and the role forced to Customer even when the request tries to send `role: "Admin"`; signing in returns 200 with a JWT; a duplicate email returns 409 and a wrong password returns 401.

Box 1 below: signUp and signIn in userController.js. **Box 2 below:** Postman 1.2 Register (201, role forced to Customer) and 1.4 Login (200, JWT returned).

Part 4: Creating the Authentication Module

Do the following (*Provide the Source code*):

- Inside your middleware folder create authentication.js file
- Create the authentication module using the example code:

Example Code:

```
const jwt = require("jsonwebtoken");
const {SECRET_KEY} = require("../../config");
const { HttpStatus } = require("../config/Constants.js")

module.exports = async (request, response, next) => {
 try {
```

```
  //   get the token from the authorization header
  const token = await request.headers.authorization.split(" ")[1];

  if (!token) {
    return response.status(HttpStatus.UNAUTHORIZED).json({
      error: new Error("Invalid Request!"),
    });
  }

  //check if the token matches the supposed origin
  const decodedToken = await jwt.verify(token, SECRET_KEY);

  // retrieve the user details of the logged in user
  const user = await decodedToken;

  // pass the user down to the endpoints here
  request.user = user;

  // pass down functionality to the endpoint
  next();
 } catch (error) {
  response.status(HttpStatus.UNAUTHORIZED).json({
    error: new Error("Invalid request!"),
  });
 }
};
```

`components/middleware/authentication.js` was created (this project already had the same logic in `authMiddleware.js` since Activity 2; it was renamed to match the file name this activity asks for, and every route file that used it was updated). It verifies the JWT from the `Authorization: Bearer <token>` header and attaches its payload to `req.user`, following the example code. The import paths are adapted to this project's structure (`components/config/config.js` for `SECRET_KEY`, `components/config/constants.js` for `HttpStatus`, CommonJS throughout as in every earlier activity) and it reuses the `sendError` helper from Activity 3 instead of building the JSON response inline, so every error from this middleware already matches the project's standard `{ success, message }` format.

```
const jwt = require("jsonwebtoken");
const { SECRET_KEY } = require("../config/config");
const { HttpStatus } = require("../config/constants");
const { sendError } = require("../utils/apiResponse");

const authentication = (req, res, next) => {
  const authHeader = req.headers.authorization;
  if (!authHeader || !authHeader.startsWith("Bearer ")) {
```

```
    return sendError(res, HttpStatus.UNAUTHORIZED, "No token provided");
  }

  try {
    req.user = jwt.verify(authHeader.split(" ")[1], SECRET_KEY); // { id, email, role }
    next();
  } catch (error) {
    sendError(res, HttpStatus.UNAUTHORIZED, "Invalid or expired token");
  }
};

module.exports = authentication;
```

`req.user` now carries `role` (from the JWT payload created in Part 3), which `authorization.js` (Part 6) reads to apply RBAC.

Screenshot below: components/middleware/authentication.js.

Part 5: Adding the Authentication Module in your User Routes

Do the following (*Provide the Source code and Output, add more pages if necessary***):**

- Import your authentication module in all your routes files.
- Test your Routes using Postman/Insomnia REST with your newly implemented Authentication.
- Make sure to add the token in your Postman/Insomnia Rest after logging in.
- Use the following example code for your reference

Example Code:

```
//UserRoutes.js
const express = require(“express”);
```

```
const router = express.Router();
const UserController = require(“../Controller/UserController.js);
const authentication = require(“../Middleware/authentication.js”);

router.get("/user/", authentication, UserController.getAllUsers);
router.get("/:Id”, authentication, UserController .getUserById);
router.post("/login", UserController .login);
router.post("/register", UserController.registerUser);
router.patch(“/update/:Id”, authentication, UserController.updateUser);
router.delete(“/remove/:Id”, authentication, UserController.deleteUser);

module.exports = router;
```

`authentication` was imported into `userRoutes.js` and every route except registration and login (which happen before a token exists) now requires a valid token. `GET /me` must come before `GET /:id` so Express does not treat "me" as an `:id` value. (Role-based `authorization` is layered on top of this in Part 7, once `authorization.js` exists.)

```
const express = require("express");
const { signUp, getProfile, getAllUsers, getUserById, updateUser, deleteUser } =
require("../controllers/userController");
const authentication = require("../middleware/authentication");

const router = express.Router();
router.post("/", signUp);
router.get("/me", authentication, getProfile);
router.get("/", authentication, getAllUsers);
router.get("/:id", authentication, getUserById);
router.patch("/:id", authentication, updateUser);
router.delete("/:id", authentication, deleteUser);
module.exports = router;
```

Method	Route	Controller function	Protected by
POST	/api/v1/users	signUp (register)	Public
POST	/api/v1/sessions	signIn (login)	Public
GET	/api/v1/users/me	getProfile	authentication
GET	/api/v1/users	getAllUsers	authentication
GET	/api/v1/users/:id	getUserById	authentication
PATCH	/api/v1/users/:id	updateUser	authentication
DELETE	/api/v1/users/:id	deleteUser	authentication

Tested in Postman (folder 1): with a valid token, `GET /api/v1/users/me` returns 200 with the signed-in user's profile. Without a token (1.7) or with a malformed one (1.8), it returns 401 "No token provided" / "Invalid or expired token" instead of the profile.

Output

[Paste screenshot here: 1.6 GET /users/me with a valid token – 200 OK]

[Paste screenshot here: 1.7 GET /users/me without a token – 401 Unauthorized]

[Paste screenshot here: 1.8 GET /users/me with a malformed token – 401 Unauthorized]

Part 6: Creating the Authentication Module

Do the following (*Provide the Source code*):

- Inside your middleware folder create authorization.js file
- Create the authorization module using the example code:

Example Code:

```
const { HttpStatus } = require("../config/Constants");

module.exports = (...roles) => {
  return async (request, response, next) => {
    try {
      if (!roles.includes(request.user.role)) {
        return response.status(HttpStatus.FORBIDDEN).json({
          error: new Error("Access denied"),
        });
      }
      next();
    } catch (error) {
      response.status(HttpStatus.FORBIDDEN).json({
        error: new Error("Access denied"),
      });
    }
  };
};
```

`components/middleware/authorization.js` applies **Role-Based Access Control**: it is called as `authorization("Admin")` (or with several roles) and rejects the request with **403 Forbidden** unless `req.user.role` — set by `authentication.js` just before it — is in the allowed list. It is used after `authentication` on every route, exactly as in the example code; `HttpStatus` and `sendError` come from this project's existing `config/constants.js` and `utils/apiResponse.js`.

```
const { HttpStatus } = require("../config/constants");
const { sendError } = require("../utils/apiResponse");

const authorization = (...roles) => {
  return (req, res, next) => {
    if (!req.user || !roles.includes(req.user.role)) {
      return sendError(res, HttpStatus.FORBIDDEN, "Access denied");
    }
    next();
  };
};

module.exports = authorization;
```

Screenshot below: components/middleware/authorization.js.

Part 5: Adding the Authorization Module in your User Routes

Do the following (*Provide the Source code and Output, add more pages if necessary***):**

- Import your authorization module in all your routes files.
- Test your Routes using Postman/Insomnia REST with your newly implemented Authorization.
- Make sure to add the token in your Postman/Insomnia Rest after logging in.
- Use the following example code for your reference

Example Code:

```
//UserRoutes.js
...
const authorize = require("../Middleware/Authorization.js");

router.get("/user/", authentication, authorize("Admin"), UserController.getAllUsers);
router.get("/:Id", authentication, authorize("Admin"), UserController .getUserById);
router.patch("/update/:Id", authentication, authorize("Admin"), UserController.updateUser);
router.delete("/remove/:Id", authentication, authorize("Admin"), UserController.deleteUser);

...
```

`authorization("Admin")` was added after `authentication` on every route that only an Admin should use: managing user accounts (list, get by id, delete — update stays open to the account's own owner too, enforced in the controller since it is not a fixed role check) and managing the product catalog (create, update, delete on products, categories, and suppliers). Reading the catalog stays public, matching the storefront use case from Activity 3.

```
// userRoutes.js
const authorization = require("../middleware/authorization");

router.get("/", authentication, authorization("Admin"), getAllUsers);
router.get("/:id", authentication, authorization("Admin"), getUserById);
router.patch("/:id", authentication, updateUser); // self-or-admin, checked inside the controller
router.delete("/:id", authentication, authorization("Admin"), deleteUser);

// productRoutes.js / categoryRoutes.js / supplierRoutes.js (same pattern)
router.post("/", authentication, authorization("Admin"), createProduct);
router.put("/:name", authentication, authorization("Admin"), updateProduct);
router.delete("/:name", authentication, authorization("Admin"), deleteProduct);
```

Method	Route	Function	Middleware chain	Why Admin-only
GET	/api/v1/users	getAllUsers	authentication, authorization("Admin")	List every account
GET	/api/v1/users/:id	getUserById	authentication, authorization("Admin")	Look up one account
PATCH	/api/v1/users/:id	updateUser	authentication (role checked in the controller)	Admin or the same user
DELETE	/api/v1/users/:id	deleteUser	authentication, authorization("Admin")	Remove an account
POST/PUT/DELETE	/api/v1/products	create/update/deleteProduct	authentication, authorization("Admin")	Manage the catalog
POST/PUT/DELETE	/api/v1/categories	create/update/deleteCategory	authentication, authorization("Admin")	Manage the catalog
POST/PUT/DELETE	/api/v1/suppliers	create/update/deleteSupplier	authentication, authorization("Admin")	Manage the catalog

Tested with both an Admin token and a Customer token (collection **Activity4-CTADWEBL**, folders 2 and 4): the Admin gets 200/201, the Customer gets **403 Forbidden – "Access denied"**, and a request with no token at all gets **401 Unauthorized** (caught by `authentication` before `authorization` even runs).

Output

[Paste screenshot here: 2.1 / 2.2 GET /users as Admin (200) and Customer (403)]

[Paste screenshot here: 4.1 / 4.2 Create a category as Customer (403) and Admin (201)]

[Paste screenshot here: 4.3 Create a product with no token (401)]

[Paste screenshot here: 4.5 Update a product as Customer (403)]

Part 6: Enhancement Activity

Do the following:

- Analyze your previous routes and determine what functions need to have an

Authorization module Format your answer in a sample table below

- Apply the Authentication and Authorization Module in each of your Routes
- Suppose a user wants to edit its information or data, add a function inside your controller that checks if the user that is trying to edits its information is the same user hence return Unauthorized/Forbidden
- Each route test it with separate user access (e.g. Admin, Customer)
- Provide Screenshots of your Source Code and Output, add more pages if necessary

The instructor's example covers `GET /` and `DELETE /:Id` on the **User** collection. Extending the same analysis to every collection and function gives the table below. The guiding rule: **account management and catalog management are Admin-only** (a Customer has no business reason to list every account or edit the product catalog); **personal resources (cart, orders, reviews, and a user's own account) belong to one signed-in user**, so a Customer may only read or create their own and an Admin may act on anyone's; **catalog browsing and reviews stay public**, matching a real storefront where anyone can look around without an account.

Collection	Function	Route	Allowed Roles
User	POST (register)	/api/v1/users	Public
User	POST (login)	/api/v1/sessions	Public
User	GET (my profile)	/api/v1/users/me	Any signed-in user (self)
User	GET (list all)	/api/v1/users	Admin
User	GET (one)	/api/v1/users/:id	Admin
User	PATCH (update)	/api/v1/users/:id	Admin, or that same user
User	DELETE	/api/v1/users/:id	Admin
Category	GET (list / one)	/api/v1/categories, /:id	Public
Category	POST / PUT / DELETE	/api/v1/categories	Admin
Supplier	GET (list / one)	/api/v1/suppliers, /:id	Public
Supplier	POST / PUT / DELETE	/api/v1/suppliers	Admin
Product	GET (list / one)	/api/v1/products, /:name	Public
Product	POST / PUT / DELETE	/api/v1/products	Admin
Cart	GET (list / one)	/api/v1/carts, /:id	Signed-in user (own cart); Admin: any
Cart	POST (create)	/api/v1/carts	Signed-in user (own cart); Admin: for any user
Order	GET (list / one)	/api/v1/orders, /:id	Signed-in user (own orders); Admin: any
Order	POST (place)	/api/v1/orders	Signed-in user (own order); Admin: for any user
Review	GET (list / one)	/api/v1/reviews, /:id	Public
Review	POST (create)	/api/v1/reviews	Signed-in user (own review); Admin: for any user

Self-edit check

Suppose a Customer tries to edit someone else's account. `updateUser` checks this before touching the

database: if the signed-in user is not an Admin and the `:id` in the URL is not their own id, it returns **403 Forbidden**. An Admin may update anyone, and `role` in the request body is silently dropped unless the requester is an Admin, so a Customer can never promote themselves by editing their own profile.

```
const updateUser = async (req, res) => {
  if (req.user.role !== "Admin" && req.user.id !== req.params.id) {
    return sendError(res, HttpStatus.FORBIDDEN, "You can only update your own account");
  }
  const updates = { ...req.body };
  if (req.user.role !== "Admin") delete updates.role; // only an Admin may change roles
  if (updates.password) updates.password = await bcrypt.hash(updates.password, SALT);
  const user = await User.findByIdAndUpdate(req.params.id, updates, { returnDocument: "after",
runValidators: true });
  ...
};
```

The same "own resource only" rule is applied to carts, orders, and reviews: `createCart`, `createOrder`, and `createReview` force the record's owner to the signed-in user (ignoring any `user` field a Customer sends in the body, unless the requester is an Admin), and `getCarts`/`getCart`/`getOrders`/`getOrder` return **403 Forbidden** if a Customer asks for a cart or order that is not their own.

Test Using Customer Role in GET api/v1/user/:id

A Customer token (Dale) calling `GET /api/v1/users/:id` for another account returns **403 Forbidden – "Access denied"** (Postman 2.5); the same request with the Admin token returns **200 OK** with the account's data (2.4).

Output — GET /api/v1/users/:id

[Paste screenshot here: 2.4 As Admin – 200 OK]

[Paste screenshot here: 2.5 As a different Customer – 403 Forbidden]

Testing with separate Admin and Customer accounts

All 46 requests of the Postman collection **Activity4-CTADWEBL** (`Activity4-CTADWEBL.postman_collection.json`) were run using two accounts: the seeded Admin (`admin@bulldogex.nu.edu.ph`) and a newly registered Customer (Dale). Every request has automated tests (status code, standard JSON format, and the expected data), and all of them pass (Test Results tab).

Self-edit enforcement

[Paste screenshot here: 3.1 Dale updates his own account – 200 OK]

[Paste screenshot here: 3.2 Dale updates the Admin's account – 403 Forbidden]

[Paste screenshot here: 3.3 Admin updates Dale's account – 200 OK]

[Paste screenshot here: 3.4 Dale tries to set role: "Admin" on himself – 200 OK but role stays Customer]

Cart ownership

[Paste screenshot here: 5.1 Dale creates his own cart – 201 Created]

[Paste screenshot here: 5.2 Dale creates a cart for someone else – 403 Forbidden]

[Paste screenshot here: 5.4 Dale requests someone else's cart by ?user= – 403 Forbidden]

[Paste screenshot here: 5.6 A different Customer (Maria) opens Dale's cart by id – 403 Forbidden]

[Paste screenshot here: 5.7 Admin opens Dale's cart – 200 OK]

[Paste screenshot here: 5.8 GET /carts with no token – 401 Unauthorized]

Order and review ownership

[Paste screenshot here: 6.1 Dale places his own order – 201 Created]

[Paste screenshot here: 6.2 Dale requests someone else's orders by ?user= – 403 Forbidden]

[Paste screenshot here: 6.4 Maria views Dale's order by id – 403 Forbidden]

[Paste screenshot here: 6.5 Dale reviews a product as himself – 201 Created]

[Paste screenshot here: 6.6 Dale tries to post a review as someone else – 403 Forbidden]

[Paste screenshot here: 6.7 GET /reviews with no token (still public) – 200 OK]

Cleanup

[Paste screenshot here: 7.1 Dale tries to delete his own account – 403 Forbidden (Admin only)]

[Paste screenshot here: 7.2 Admin deletes Dale's account – 200 OK]

Conclusion

The Bulldogs Exchange API now authenticates every request with a JWT that carries the user's role, and authorizes each route with that role: Admin-only account and catalog management, owner-or-admin access to personal resources (cart, orders, reviews, and a user's own profile), and public read access to the storefront. Password hashing with bcrypt, JWT-based sessions, protected routes, and Role-Based Access Control were all verified in Postman using separate Admin and Customer accounts, including the cases meant to fail (no token, wrong role, editing someone else's data), which consistently returned 401 or 403 instead of leaking or changing data they should not.

Activity Rubrics

Criteria	Beginning (1)	Developing (2)	Proficient (3)	Excellent (4)	Pts
User Registration and Password Security	Registration is incomplete or does not function correctly. Passwords are stored insecurely or Bcrypt is not implemented.	Registration functions partially, but contains errors in validation, user creation, or password hashing.	Registration works correctly and passwords are hashed using Bcrypt, with only minor issues.	Registration is fully functional with appropriate validation and secure Bcrypt password hashing implemented correctly.	
User Login and JWT	Login and JWT authentication are	Login and JWT generation are	Login and JWT authentication	Login, credential verification, JWT	

Authentication	not implemented or fail to authenticate users correctly.	partially implemented but contain errors in credential verification, token generation, or handling.	function correctly with minor issues that do not significantly affect functionality.	generation, and token-based authentication are fully implemented and function correctly.	
Protected API Routes and Authentication	Protected routes are not implemented or unauthenticated users can access protected resources.	Some routes are protected, but authentication checks are inconsistent or contain errors.	Required API routes are protected and valid JWTs are properly verified, with minor issues.	All designated routes are properly protected, JWTs are correctly verified, and unauthenticated requests are consistently rejected with appropriate responses.	
Role-Based Access Control and Permission Management	Roles or permissions are not implemented, or users can access resources regardless of their assigned roles.	Roles are partially implemented but permission restrictions contain several errors or inconsistencies.	Admin and Customer roles are implemented with appropriate access restrictions and only minor issues.	RBAC is fully implemented with clearly defined permissions, and each role can access only the resources permitted to it.	
Testing, Documentation, and Submission	Required testing, screenshots, and documentation are largely missing or incomplete.	Some authentication and authorization scenarios are tested, but several required tests or documentation are missing.	Required authentication, authorization, and permission scenarios are tested and documented with only minor omissions.	All required scenarios are systematically tested using Postman, results are clearly demonstrated, and all screenshots, source code, and documentation are complete and well-organized.	
Total Score					